



• PRODUCT CASE STUDY · V2

WordPress blogging on Autopilot.

How a weekend Streamlit script for auto-posting to WordPress grew into a self-hosted, multi-tenant blogging agent with its own memory of your writing style.

AUTHOR	TIMELINE	STATUS	WEBSITE
Vinamra · vinamra.dev	July 2026 – Present	Live · Auto-Pilot Active	blogassist.vinamra.dev
FastAPI APScheduler SQLAlchemy · SQLite JWT Auth Vanilla JS + Tailwind			
github.com/vinamrapandey/Blog-Assist			

Content creation doesn't scale by itself.

Writing high-quality, SEO-optimized blog posts consistently requires real time, effort, and domain expertise — and it's the first thing that gets dropped when an agency or a solo founder gets busy. The obvious digital shortcuts don't quite fit the job either.



Raw LLMs

Powerful, but they hand you a wall of text. Someone still has to copy it, format it, and log into WordPress to upload it — every single time.



Heavy SaaS Writers

Bloated dashboards, recurring subscriptions, and your content and credentials living on someone else's server instead of yours.

"BlogAssist Pro starts from a different premise: configure it once, and let it run forever in the background." — it keeps the workflow a blogger already trusts (drafting, editing, WordPress publishing) but removes the subscription, the manual babysitting, and the learning curve.



Self-Hosted, Not Rented

Runs on infrastructure you control. Your API keys, your WordPress credentials, and your generated content never leave your own database.



Zero Daily Touch

Set a topic and an interval — anywhere from once a minute (for testing) to once a week — and the agent takes it from there.

A self-hosted engine built for hands-free content.

BlogAssist Pro pairs a decoupled FastAPI backend with a Vanilla JS + Tailwind frontend around one goal: generate, review, and publish blog content without anyone sitting at a keyboard.

WHAT THE USER SEES



Live Dashboard

Total-posts counter, a Weekly Activity chart of real published posts, and a pulse-animated "Auto-Pilot Active" badge with a live countdown to the next run.



Preview & Critique

Every draft can be reviewed in an editable modal — quick-critique chips, free-text notes, then "Rewrite & Learn" or "Approve & Publish."



AI Brain

A running list of the style rules the agent has learned from feedback — readable, and deletable one by one.



Activity Log

A live, scrolling terminal of every background action, polled in real time for full transparency.

HOW IT COMPARES

FEATURE	BLOGASSIST PRO	MANUAL WORDPRESS	SAAS AI WRITERS
Fully Autonomous	Yes	No	Rarely
Data Ownership	Local / Self-hosted	Yes	Cloud Vendor
Background Scheduling	Yes (APScheduler)	No	Varies
Learns Your Style	Yes (Feedback → Rule)	N/A	Rarely
Multi-User Ready	Yes (JWT accounts)	No	Yes

From a personal script to a multi-tenant product.

The original build proved the concept for one user, on one machine. The current codebase turns that into an actual product surface — accounts, a real landing page, and a feedback loop the agent can learn from.



Accounts & Multi-Tenancy

JWT-based login and registration (OAuth2 password flow, bcrypt hashing). Every Config, Log, Post, and StyleGuideline row is scoped to a `user_id`, and each user's scheduler job runs under its own namespaced job ID (`auto_post_{user_id}`), so one server can safely run many agents in parallel.



Guided Onboarding

A three-step wizard — Connect your AI, Connect WordPress, Define Strategy — replaces a blank settings page for first-time users, with credentials validated live before the account is marked ready.



Style Feedback Loop

Critiquing a draft doesn't just rewrite it. The same call asks the model to extract one generalized, under-15-word style rule, which is stored and silently re-injected into every future prompt.



Public Landing Page

A proper marketing front door — hero, feature grid, and a "Get Started" flow — sitting in front of the authenticated app, in the same orange-and-black visual language as the dashboard.

Multi-tenant support was on the "Next" list in the original roadmap. It's now shipped: the scheduler, the database models, and the auth layer were all rebuilt around `user_id` as a first-class citizen rather than bolted on afterward.

Four calls that shaped the architecture.

1

Decoupled FastAPI + Vanilla JS over Streamlit

The first version was a Streamlit script; it proved the idea but couldn't run true background jobs. Every screen now talks to a REST API instead. Trading React/Vue for plain HTML/JS was deliberate — zero build step, instant iteration.

2

APScheduler with Thread-Safe DB Sessions

Background loops crash easily on detached SQLAlchemy instances. Config values are extracted into plain variables before they're closed over by the scheduler thread, and a FastAPI startup hook re-arms any agent that was mid-run when the server restarted.

3

Zero-Plugin WordPress Auth

Instead of shipping a custom WP plugin, the agent authenticates with native WordPress Application Passwords over Basic Auth — one less moving part to install, update, or break.

4

Prompt-Injected Style Memory over Fine-Tuning

Rather than fine-tuning a model per user, feedback is distilled by the LLM itself into a short rule and stored as plain rows. Every future prompt replays the accumulated rules — personalization with no training cost.

From MVP to a multi-tenant product.

The project's fix log reads like a running journal — one issue, one status, one action per entry — the exact record an agentic coding tool leaves behind while working through a punch list.

● PHASE 1	Streamlit MVP Basic LLM → WordPress connectivity	DONE
● PHASE 2	FastAPI Migration Vanilla JS frontend with Tailwind	DONE
● PHASE 3	Scheduling Hardening APScheduler + DetachedInstanceError fix	DONE
● PHASE 4	UI Overhaul Countdown timers, Auto-Pilot active states	DONE
● PHASE 5	Multi-Tenant + AI Brain JWT accounts, onboarding wizard, style feedback loop, landing page	DONE
● NEXT	Image Generation + Advanced SEO Auto feature images, meta descriptions, internal links	PLANNED

5

DB Models — User, Config, Log, Post, Guidelines

10

Configurable Settings

4

Agent Controls

24/7

Autonomous Runtime

1m–168h

Schedule Range

2

LLM Providers + Simulated Mode

Built with real agentic help, end to end.

This project leaned on agentic coding tools for both implementation and debugging, while every product decision traced back to a concrete workflow need.

HUMAN

Ideation & Product Decisions

Choosing WordPress Application Passwords over a plugin, decoupling from Streamlit, adding a "fire and forget" countdown timer, and later — rebuilding the whole thing around accounts — all came from the builder's own workflow, not a template.

AI

Implementation — Google Antigravity

Google's agentic IDE handled the architectural migration: writing the FastAPI backend, styling the Vanilla JS frontend, diagnosing SQLAlchemy DetachedInstanceErrors from raw log traces, and later implementing the JWT auth layer and the feedback-to-rule extraction logic.

WHAT THE AI EXCELLED AT

- Refactoring a monolithic Streamlit script into a decoupled, multi-tenant API architecture.
- Diagnosing specific ORM threading bugs from log traces and fixing them directly.
- Translating UI requirements into working Vanilla JS logic — the live countdown timer, the contenteditable draft preview, the polling activity log.
- Handling Git operations, .gitignore hygiene, and clean, incremental commits autonomously.

WHAT'S NEXT

Image Generation

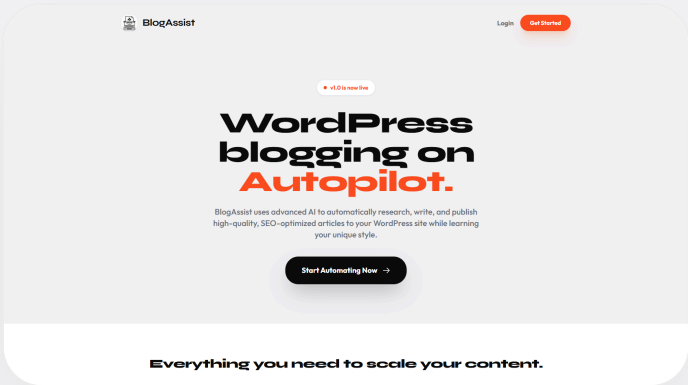
Auto-attach DALL·E or Midjourney feature images to every WordPress post.

Advanced SEO

Auto-generated meta descriptions, tags, and internal linking between posts.

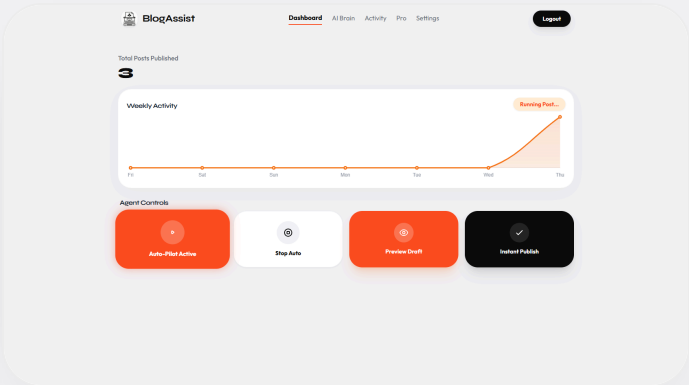
The interface, in the flesh.

Captured on the live BlogAssist deployment — fully responsive, glassmorphic, orange-and-black throughout the authenticated app.



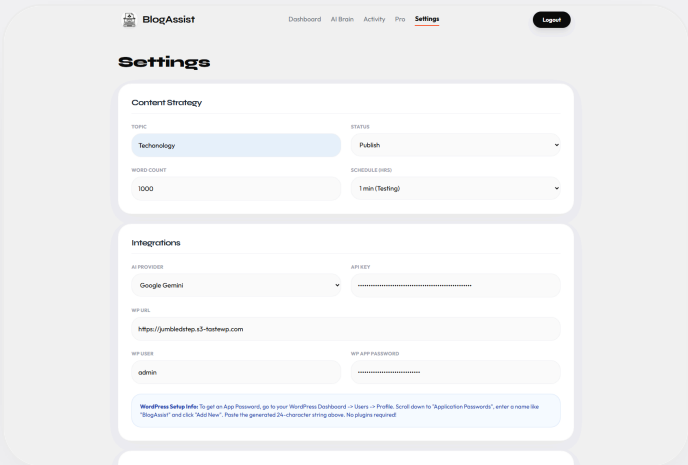
Landing Page

Marketing front door



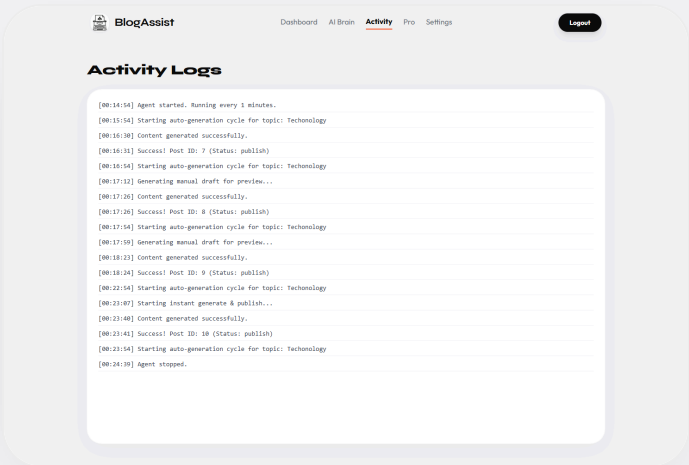
Dashboard (Home)

Chart • Auto-Pilot • controls



Settings

AI provider • WordPress • schedule

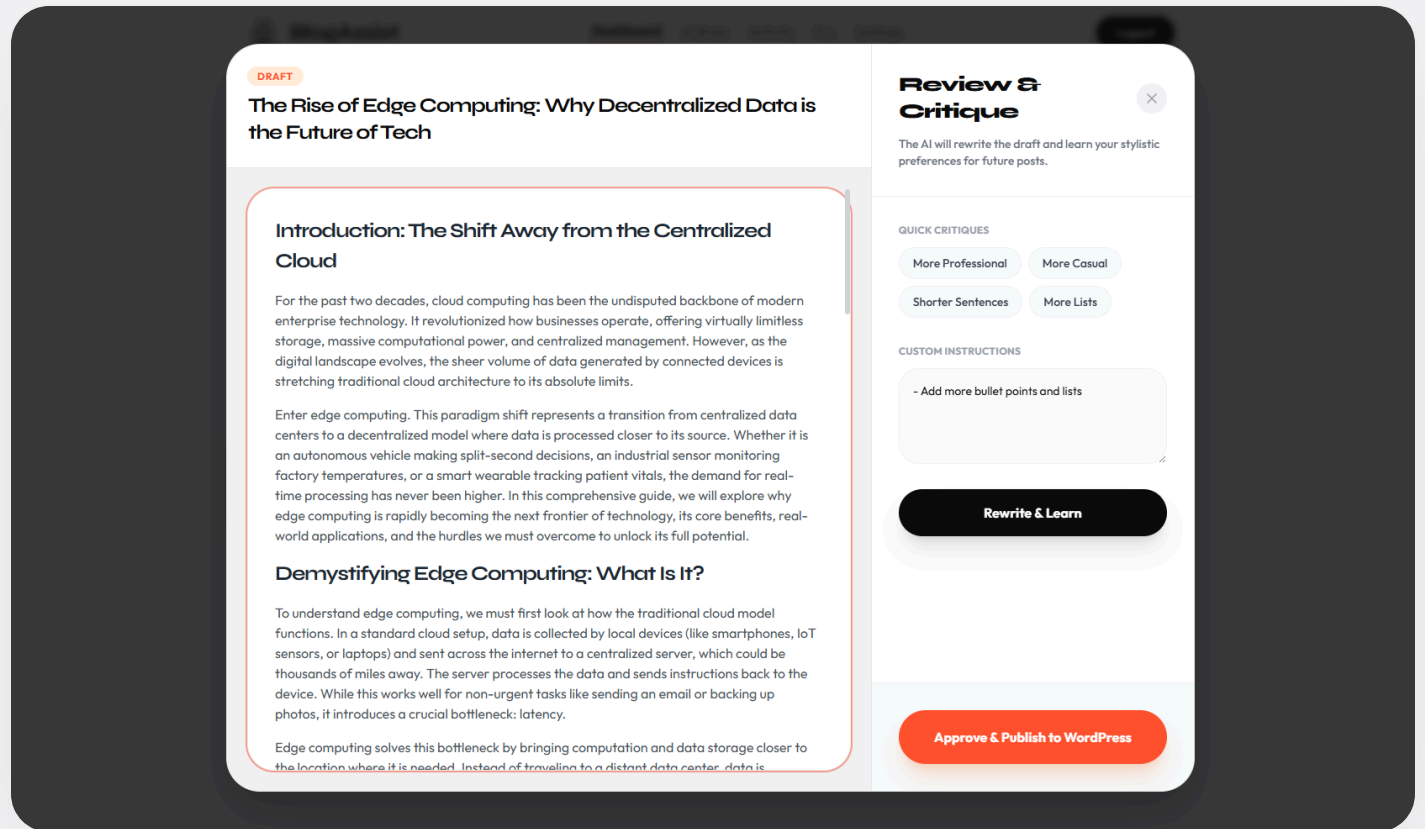


Activity Log

Live scrolling terminal feed

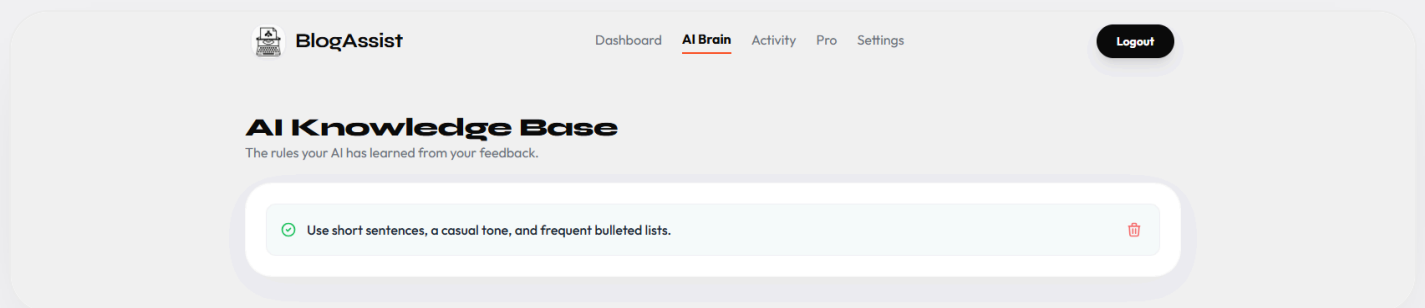
Watching the agent learn.

The two screens that make BlogAssist feel less like a script and more like a collaborator: a reviewable draft, and a growing memory of how you like things written.



Preview & Critique Modal

Editable title/body · quick critiques · custom instructions



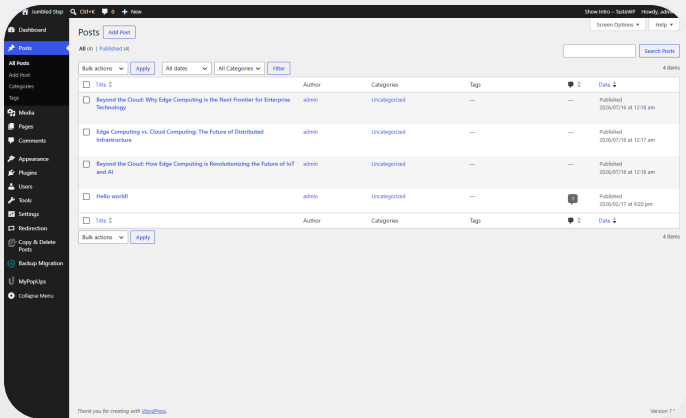
AI Brain

Every rule learned from a "Rewrite & Learn" — one row per rule, deletable

In this run, one round of feedback ("more casual, more lists") was enough for the agent to permanently learn: "Use short sentences, a casual tone, and frequent bulleted lists." — visible proof the RAG-lite loop from Decision 4 is working end to end.

From agent to a live WordPress post.

The full loop, closed: the agent runs on its schedule, the post lands in WordPress with the right status, and it renders on the live site — no manual copy-paste anywhere in between.



WordPress • Posts 4 posts published autonomously

"Beyond the Cloud: Why Edge Computing is the Next Frontier for Enterprise Technology" — drafted, formatted with real headings and bullet lists, and pushed live to **jumbledstep.s3-tastewp.com** entirely by the scheduled agent.

- **Bandwidth Cost Optimization:** Transmitting vast amounts of raw data, such as high-definition surveillance videos, across wide-area networks (WAN) is prohibitively expensive. Processing video feeds locally at the camera level (the edge) and only uploading relevant clips or metadata significantly lowers telecommunication costs.
- **Data Sovereignty and Regulatory Compliance:** Strict regulatory frameworks such as GDPR in Europe and CCPA in California dictate where personal data can be stored and processed. Edge computing enables organizations to keep sensitive data within local jurisdictions, ensuring compliance without sacrificing analytical capabilities.

The Convergence of Edge Computing and Artificial Intelligence (Edge AI)

Perhaps the most transformative development in modern enterprise tech is the convergence of edge computing and artificial intelligence, commonly referred to as **Edge AI**. Traditionally, running complex machine learning models required the massive parallel processing power of cloud-based GPUs. However, advancements in specialized silicon—such as neural processing units (NPUs) and application-specific integrated circuits (ASICs)—have made it possible to run sophisticated inference models on low-power edge devices.

Edge AI allows devices to perform real-time image recognition, natural language processing, and anomaly detection locally. For instance, a smart security camera can identify a security breach and trigger an alarm entirely offline, without needing to ping a cloud server to recognize the threat. This localized intelligence makes automated systems faster, more secure, and highly resilient to network outages.

Industry Use Cases: Edge Computing in Action

Edge computing is no longer a theoretical concept; it is actively delivering tangible business value across a multitude of industries today.

1. Manufacturing and Industry 4.0

In modern smart factories, thousands of sensors monitor vibration, temperature, and performance metrics on production lines. By deploying edge gateways directly on the factory floor, manufacturers can run predictive maintenance models that detect microscopic anomalies in machinery before a breakdown occurs. This real-time detection minimizes unplanned downtime, which can cost

Live Published Post Mid-article — formatted & published unattended

Links & contact.

WEBSITE	blogassist.vinamra.dev
REPOSITORY	github.com/vinamrapandey/Blog-Assist
LICENSE	MIT
CONTACT	vinamra.dev

**BlogAssist Pro is a working answer to a
specific
problem — built, hardened, and now
running on autopilot.**